

Downloading Data by Spamming `java.net.URL#equals(URL)`

Will

Abstract—Checking whether a URL equals another URL in Java causes a DNS resolution to happen. What a beautiful language. Anyway with a cooperative DNS server we can transfer data this way.

I. Introduction

URLs. What are they? I don't know. Apparently it's a type of URI, but that raises even more questions.

Anyway, if you're really bored you might look at two URLs and think "are they the same?"

Your criteria for 'the same' might not be exact - you might say that a trailing slash doesn't matter. But you're unlikely to say that `https://ahadhj.example/a/b/c` and `https://dfjkghsld.example/a/b/c` are the same just because `ahadhj.example` and `dfjkghsld.example` happen to resolve to the same IP address.

Java decides that that's how it should work though. So doing `url1.equals(url2)` allows us to get 1 (one) bit (bit) of data from our DNS server. So if we spam it, we can get more bits! And it turns out all data is just More Bits.

II. Figs



Fig. 1. It looks tasty.

III. what did you do

I wrote a program that spams `URL#equals` on some URLs. if `http://data.internal.equals http://one.internal`, it records a one. If it equals `http://zero.internal`, it records a zero. there's also bit- and byte- end markers for synchronisation.

My first implementation just overrode the JVM's address resolver, but that felt like cheating. No actual DNS was happening. So I wrote a "DNS server" and used that instead.

I ran the DNS server on my computer, and then made a virtual machine whose DNS config was set to only talk to my DNS server.

```
var one = URI.create("http://one.internal").toURL();
var bitMarker = URI.create("http://bit.internal").toURL();
var byteMarker = URI.create("http://byte.internal").toURL();
while (this.idx < this.data.length) {
    var data = URI.create("http://data.internal").toURL();
    if (this.idx == -1) {
        if (data.equals(byteMarker)) {
            this.idx = 0;
        }
    } else if (data.equals(byteMarker)) {
        if (this.lastBit != -1) {
            this.incrementByteIdx();
            this.lastBit = -1;
        }
    } else if (data.equals(bitMarker)) {
        if (this.lastBit != -1) {
            this.incrementIdx();
            this.lastBit = -1;
        }
    } else {
        boolean bit = data.equals(one);
        if (this.lastBit == -1) {
            this.setBit(bit);
        } else {
            if ((this.lastBit == 1) != bit) {
                System.err.println("we missed a bitMarker. uh oh");
            }
            this.incrementIdx();
            if (this.idx < this.data.length) {
                this.setBit(bit);
            }
        }
    }
}
```

Listing 1. The only bit of the code that matters

IV. does it work

Amazingly yes. You do need to disable all your dns caching of course. But it works!

I could get it to work well at 2 baud. I tried increasing that to 10 baud and it wasn't very reliable. It is quite slow though. lol.

Also there isn't any proper way of marking the start of the message (byte-wise) so you've got to work that out yourself when you read the output.

I'll throw some extra info and the full code onto <https://sting.willbl.dev/sigbovik26/index.html>.

```
received bit 1  
received bit marker  
received bit 0  
received bit marker  
received bit 0  
received byte marker  
lo    hel  
sigbovik@sigbovik2026:~$
```

Fig. 1. The result. Due to a lack of message start synchronisation it would appear that the 'hello' message became some sort of Norse religious proclamation.



Fig. 2. Lol Hell

V. but like what else can i do

It would be funny if you implemented DNS on top of this I think.